DESIGN AND ANALYSIS OF ALGORITHMS

DOMINATING SET PROBLEM

An NP-Complete Optimization Problem

Name

Paleru Dharani Govardhan

RegNo

192525280

Course

Design and Analysis of Algorithms

Topic

Dominating Set Problem (NP-Complete Problem)

Report Type

Technical Report — Algorithm Design, Backtracking, Pruning, and Complexity Analysis

1. Introduction

1.1 Graphs, Vertices, and Edges

A graph is a mathematical structure used to model pairwise relationships between objects. Formally, a graph is written as G = (V, E), where V is a finite set of vertices (also called nodes) and E is a set of edges, each edge being an unordered pair of vertices. Vertices typically represent discrete entities such as sensors, routers, people, or locations, while edges represent a relationship or connection between two such entities, such as a communication link, a friendship, or physical adjacency.

Graphs are the natural modelling tool whenever a problem concerns connectivity, coverage, or influence between discrete entities. The Dominating Set Problem, the subject of this report, is one of the most studied graph-covering problems in combinatorial optimization and theoretical computer science.

1.2 What Domination Means in a Graph

Domination is a coverage relationship. A vertex v is said to dominate itself and every vertex directly connected to it by an edge. A subset of vertices D ⊆ V is called a dominating set if every vertex of the graph is either a member of D or is adjacent to at least one member of D. Intuitively, the vertices in D act as “centres” that reach out along their edges to cover their neighbours, and the union of everything reached must be the entire vertex set.

This idea appears naturally whenever a small number of resources must be placed so that they collectively cover, monitor, or serve an entire network. A single guard cannot watch an entire building, but a well-chosen small set of guards, each watching their own room and the rooms adjacent to it, can watch the whole building.

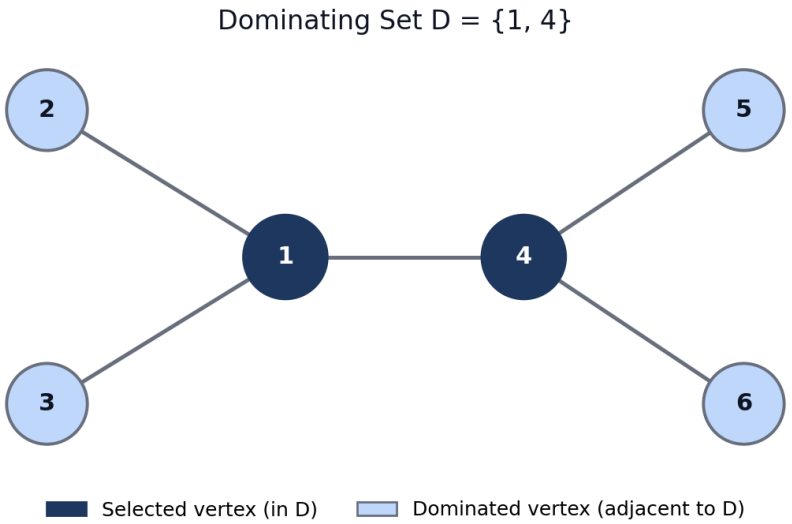


Figure 1: Example of a Dominating Set in a Graph. Vertices 1 and 4 are selected (dark); every remaining vertex is adjacent to a selected vertex and is therefore dominated (light blue).

1.3 The Dominating Set Problem as an Optimization Problem

Many different subsets of V may satisfy the domination condition — in the most extreme case, the full vertex set V is always a (trivial) dominating set, since every vertex trivially dominates itself. The interesting question is not whether a dominating set exists, but how small a dominating set can be found. The Minimum Dominating Set Problem asks for a dominating set D of the smallest possible cardinality |D|. This turns a

feasibility question into an optimization question: among all feasible (dominating) subsets, select the one that minimizes the objective function |D|.

1.4 Why the Problem Is Computationally Challenging

The number of candidate subsets of an n-vertex graph is 2^n, since every vertex can independently be included or excluded from a candidate set. Checking a single candidate for feasibility (whether it dominates every vertex) can be done efficiently, but there is no known method for directly identifying the minimum-size feasible subset without, in the worst case, implicitly considering an exponential number of candidates. This combinatorial explosion is the source of the problem's computational hardness, and it places the Dominating Set Problem within the class of NP-Complete problems, discussed formally in Section 9. The remainder of this report develops a precise mathematical definition of the problem, an exact backtracking algorithm with pruning, a correctness argument, and a complexity analysis.

2. Formal Problem Definition

Let G = (V, E) be a finite, undirected, simple graph, where V is the vertex set and E ⊆ V × V is the edge set. A subset D ⊆ V is called a dominating set of G if the following condition holds:

$$\forall\, v \in V:\ \ v \in D \ \ \ OR \ \ \ \exists\, u \in D \text{ such that } (u, v) \in E$$

The objective of the optimization version of the problem is:

$$\text{Minimize } |D|, \ \text{subject to } D \text{ being a dominating set of } G$$

2.1 Closed Neighbourhood

A convenient and standard way to express the domination condition uses the closed neighbourhood of a vertex. The open neighbourhood N(v) of a vertex v is the set of vertices adjacent to v. The closed neighbourhood adds the vertex itself:

$$N[v] = \{v\} \cup N(v)$$

Using this notation, the domination condition can be restated compactly: D is a dominating set of G if and only if

$$\forall\, v \in V, \ \ N[v] \cap D \neq \emptyset$$

This states that for every vertex v, its closed neighbourhood must contain at least one vertex from D — either v itself is chosen, or one of its neighbours is chosen.

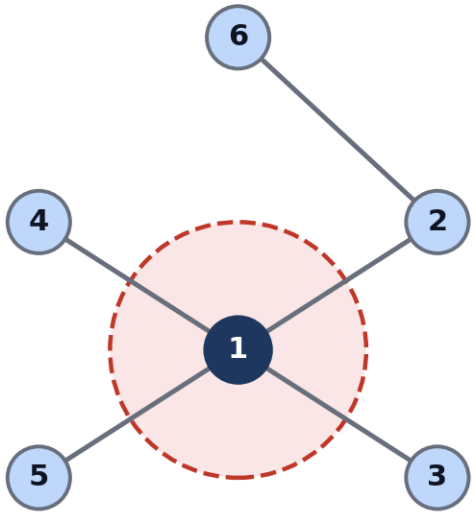


Figure 3: The closed neighbourhood N[1] = {1,2,3,4,5} of vertex 1. Vertex 6 lies outside N[1] and is therefore not dominated by vertex 1 alone.

2.2 Key Terminology

Term	Meaning
Graph G = (V, E)	The undirected graph on which domination is defined.
Vertex set V	The finite set of nodes to be dominated.
Edge set E	The set of adjacency relationships between vertex pairs.
Dominating set D	A subset of V satisfying the domination condition.
Minimum dominating set	A dominating set of the smallest possible size, denoted γ(G).
Dominated vertex	A vertex that is in D or adjacent to a vertex in D.
Undominated vertex	A vertex that is neither in D nor adjacent to any vertex in D.

Table 1: Basic Graph and Domination Terminology.

The minimum possible size of a dominating set of G is called the domination number, commonly denoted γ(G). Computing γ(G) exactly for an arbitrary graph is the central computational challenge addressed in this report.

2.3 Dominating Set versus Non-Dominating Set

Figure 2 illustrates the distinction directly on a single example graph. In part (a), the set D = {1, 4} satisfies the domination condition: vertices 2 and 3 are adjacent to vertex 1, and vertices 5 and 6 are adjacent to vertex 4, so every vertex is either selected or dominated. In part (b), the set D = {2, 5} fails the condition, because vertices 3 and 6 are adjacent only to vertex 1 and vertex 4 respectively, neither of which is selected nor is itself in D — so vertices 3 and 6 remain undominated and the set is not a valid dominating set.

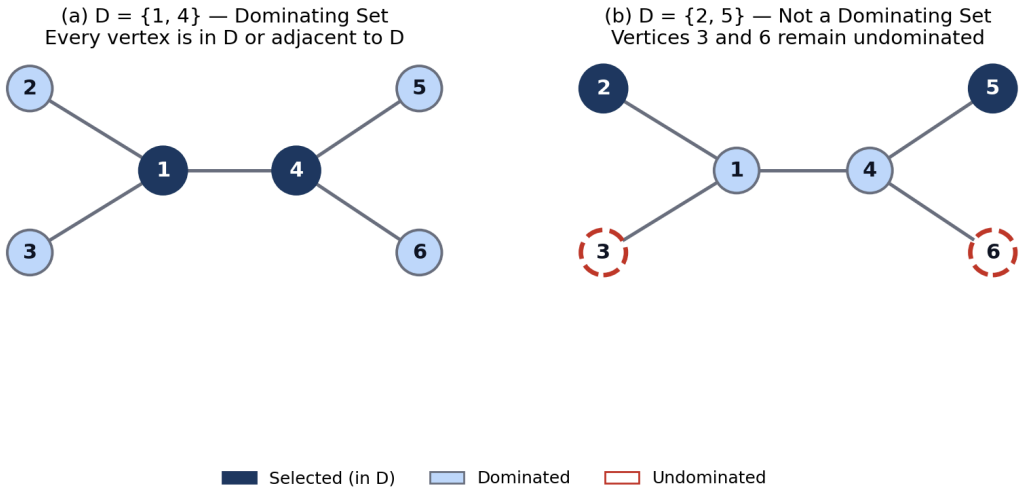


Figure 2: Dominating set (a) versus non-dominating set (b) on the same graph. Dashed red outlines mark vertices left undominated.

3. Constraints and Feasible Solutions

Before an algorithm can search for a minimum dominating set, the constraints that define a valid (feasible) candidate must be stated precisely. These constraints determine which of the 2^n subsets of V are acceptable answers to the problem, and they are the rules that a backtracking search must respect and exploit.

3.1 Constraints of the Dominating Set Problem

1. Every vertex of the graph must be dominated: it must lie in D or be adjacent to some vertex in D.
2. A selected vertex automatically dominates itself.
3. A selected vertex also dominates every vertex adjacent to it (its open neighbourhood).
4. Every vertex must have at least one dominating relationship — no vertex may be left uncovered.
5. The candidate set D must be a subset of the vertex set: $D \subseteq V$.
6. The objective is to minimize the cardinality $|D|$ among all sets that satisfy the domination condition.
7. Each vertex may appear in D at most once; duplicate selections are meaningless in a set.
8. A candidate solution is feasible if and only if every vertex of V is dominated; partial coverage is not acceptable.

3.2 Feasible versus Infeasible Solutions

A feasible solution is any subset $D \subseteq V$ that satisfies the domination condition for every vertex in the graph, regardless of whether it is minimum in size. An infeasible solution is a subset that leaves at least one vertex undominated. The search for the optimal answer is therefore a two-stage filtering process: first, eliminate every infeasible candidate; second, among the surviving feasible candidates, retain the one(s) of minimum size.

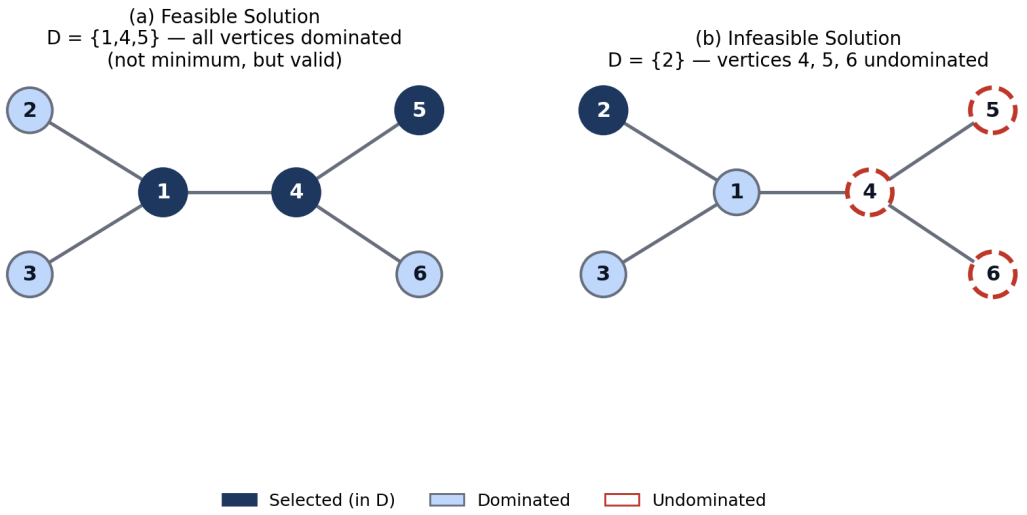


Figure 5: A feasible (but non-minimum) solution (a) compared with an infeasible solution (b) that leaves vertices undominated.

3.3 Constraint Table

Constraint	Meaning	Effect on Search Space
Full coverage	Every vertex must be dominated	Eliminates subsets that skip any vertex's neighbourhood
Self-domination	A chosen vertex covers itself	Simplifies coverage bookkeeping
Neighbour coverage	A chosen vertex covers all neighbours	Encourages high-degree vertices to be chosen early
$D \subseteq V$	Only real vertices may be chosen	Bounds the search to exactly 2^n candidates
Minimality	Smallest feasible $\|D\|$ is required	Forces comparison against a running best solution
No duplicates	Sets, not multisets	Keeps state representation as a simple bit-vector

Table 2: Dominating Set Constraints and Their Effect on the Search Space.

3.4 How Constraints Restrict the Feasible Region

Out of the 2^n subsets of an n-vertex graph, only a fraction satisfy the full-coverage constraint. As vertices are progressively included or excluded during a search, the domination constraint can be checked incrementally: each time a vertex is included, the set of currently undominated vertices shrinks by that vertex's closed neighbourhood. This incremental view is exactly what a backtracking search exploits, and it is also the basis for the pruning rules introduced in Section 6.

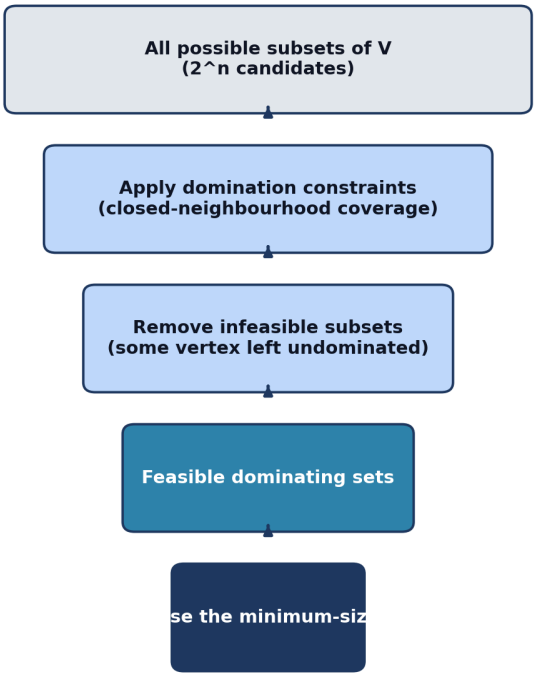


Figure 4: Constraint filtering reduces the exponential space of raw subsets down to the single minimum-size dominating set.

Category	Description	Example (Figure 5 graph)
Feasible, non-minimum	All vertices dominated, but |D| not minimal	D = {1,4,5}, |D| = 3
Feasible, minimum	All vertices dominated with smallest |D|	D = {1,4}, |D| = 2
Infeasible	At least one vertex left undominated	D = {2}, vertices 4,5,6 undominated

Table 3: Feasible versus Infeasible Solutions, Illustrated on the Running Example.

4. Backtracking Approach

Because the Dominating Set Problem is NP-Complete (Section 9), no polynomial-time algorithm is known that is guaranteed to find the exact minimum dominating set on every graph. When an exact optimum is required — for example on small or moderately sized graphs, or as a baseline against which heuristics are measured — backtracking is a natural and systematic exact technique. It explores the space of candidate subsets while abandoning (pruning) any partial candidate that cannot possibly lead to an improved feasible solution.

4.1 Candidate Solution Representation

Each candidate solution is represented as a binary decision vector of length n, one bit per vertex:

$$x\_i = 1 \ \text{if vertex } v\_i \text{ is selected } (v\_i \in D); \quad x\_i = 0 \ \text{otherwise}$$

A complete assignment of all n bits corresponds to one specific subset of V. The backtracking algorithm builds this vector one position at a time, and at each position it explores both possible values before moving on.

4.2 Include / Exclude Decisions

At every vertex v_i, the algorithm branches into two recursive calls: one in which v_i is included in the current candidate set, and one in which it is excluded. This binary branching is the defining structure of the backtracking search tree — each internal node of the tree corresponds to a decision about one vertex, and each root-to-leaf path corresponds to one complete candidate subset.

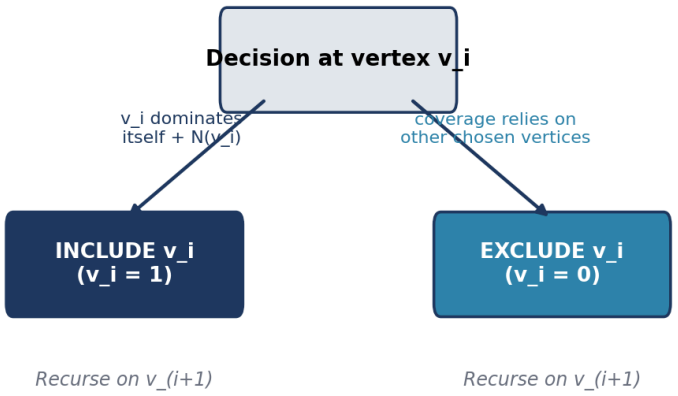


Figure 8: The include/exclude decision made at each vertex during the backtracking search.

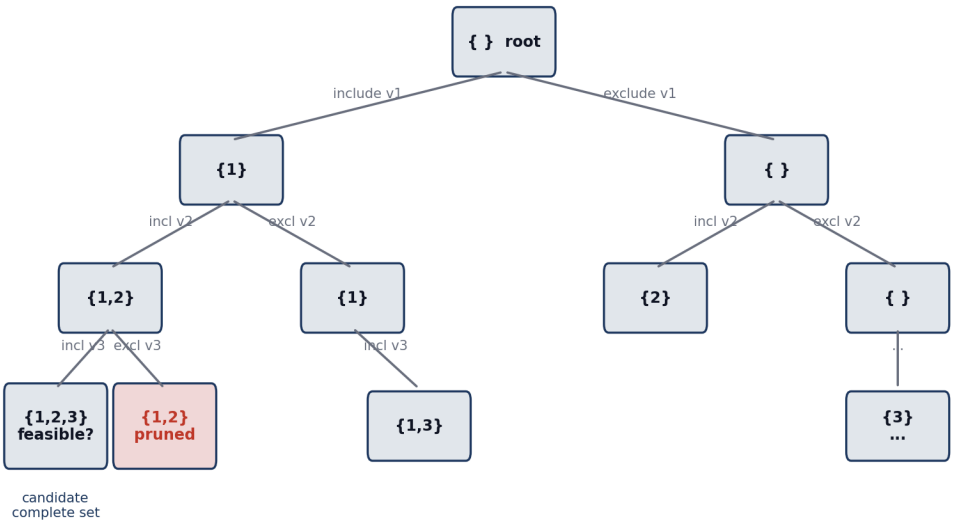
4.3 Recursion, State, and Feasibility Checking

The recursive state carried through the search consists of: the index of the vertex currently being decided, the set of vertices selected so far, and the set of vertices already dominated by that selection. At the base case — when every vertex has been decided — the algorithm checks whether the accumulated selection dominates all vertices of the graph. If it does, and if its size is smaller than the best solution found so far, the best solution is updated. The algorithm then backtracks: it undoes the most recent decision and explores the alternative branch, continuing until the entire search tree (subject to pruning) has been explored.

4.4 Backtracking Search Tree

Figure 6 shows the first few levels of the backtracking search tree for a graph on vertices v1, v2, v3, v4. The root corresponds to the empty candidate set. Each edge of the tree represents an include or exclude decision for the next vertex, and each node represents the partial candidate set accumulated along the path from the root.

Figure 6: Backtracking Search Tree for Dominating Set (vertices v1..v4)



... recursion continues for v4, generating every include/exclude path ...

Figure 6: Backtracking search tree for the Dominating Set Problem showing include/exclude branching over vertices v1..v4.

5. Backtracking Algorithm and Pseudocode

5.1 Feasibility Check Pseudocode

```
FUNCTION isDominating(D, V, adjacency):
    dominated = empty set
    FOR each vertex u in D:
        dominated.add(u)
        FOR each neighbour w of u:
            dominated.add(w)
    RETURN dominated == V
```

This routine walks through the closed neighbourhood of every selected vertex and accumulates the set of vertices reached. If, after processing all of D, every vertex of the graph has been reached, D is a valid dominating set. This check runs in O(n + m) time, where n = |V| and m = |E|, since every vertex and edge incident to a selected vertex is inspected at most once.

5.2 Basic Backtracking Pseudocode

```
BACKTRACK(index, selectedSet):
    IF index == n:                      // all vertices processed
        IF isDominating(selectedSet):
            IF |selectedSet| < |bestSolution|:
                bestSolution = selectedSet
        RETURN

    // Branch 1: INCLUDE vertex[index]
    selectedSet.add(vertex[index])
    BACKTRACK(index + 1, selectedSet)
    selectedSet.remove(vertex[index])         // undo

    // Branch 2: EXCLUDE vertex[index]
```

```
    BACKTRACK(index + 1, selectedSet)
```

5.3 Explanation of Each Step

Component	Explanation
Input	Graph G = (V, E), with vertices indexed 0..n-1
Output	A minimum dominating set bestSolution ⊆ V
Base condition	index == n: every vertex has been assigned a decision
Recursive case 1	Include the current vertex, recurse, then undo (backtrack)
Recursive case 2	Exclude the current vertex and recurse
Feasibility check	isDominating(selectedSet) tests full coverage at a leaf
Best-solution update	Replace bestSolution whenever a smaller feasible set is found

Table 4: Explanation of Each Component of the Backtracking Algorithm.

5.4 Algorithm Flowchart

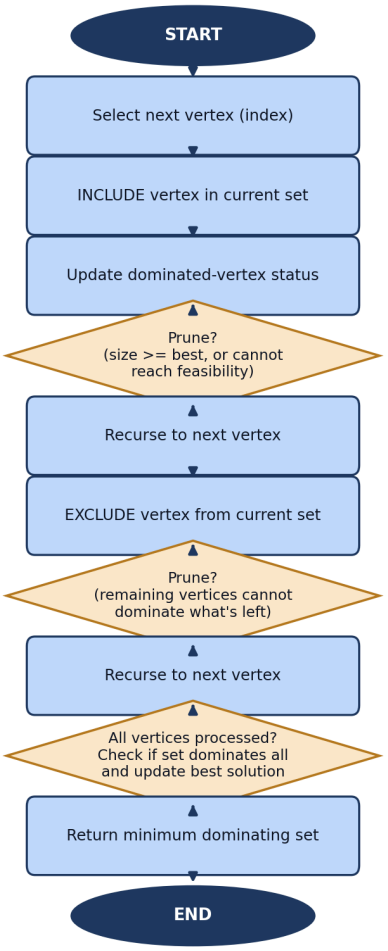


Figure 7: Flowchart of the backtracking algorithm for the Dominating Set Problem, including the two pruning decision points introduced in Section 6.

6. Pruning Techniques

Pruning is the mechanism that makes backtracking practical: it allows large portions of the search tree to be discarded without being explicitly visited, whenever it can be proven that no leaf in that portion of the tree can improve on the best solution found so far. Pruning does not change which subsets are valid dominating sets — it only changes how quickly the search can find and confirm the minimum one.

6.1 Pruning Rule 1 — Size Bound

If the size of the current partial selection already reaches or exceeds the size of the best complete feasible solution found so far, the branch cannot possibly produce an improvement, and it is abandoned immediately:

$$\textit{if } |\textit{selectedSet}| \geq |\textit{bestSolution}|\textit{: prune}$$

6.2 Pruning Rule 2 — Reachability Bound

At any point in the search, some vertices remain undominated. If the vertices that have not yet been decided cannot, even if all of them were included, cover every remaining undominated vertex, then no completion of the current branch can be feasible, and the branch is pruned:

$$\textit{if (undominated vertices)} \cap \textit{(closed neighbourhoods of all remaining candidate vertices)} \neq \textit{undominated vertices: prune}$$

6.3 Pruning Rule 3 — Early Acceptance

If, before all vertices have been explicitly decided, the current selection already dominates every vertex of the graph, the remaining undecided vertices need not be included at all (including them could only increase $|D|$). The algorithm updates the best solution immediately and stops exploring further inclusions along that branch.

6.4 Pruning Rule 4 — Lower-Bound Reasoning

A simple combinatorial lower bound can also be used: since one vertex dominates at most (1 + its degree) vertices, the number of additional vertices still required to dominate the remaining undominated vertices is at least $\lceil$(undominated count) / (1 + maximum degree among remaining vertices)$\rceil$. If the current selection size plus this lower bound is not smaller than the best known solution, the branch is pruned.

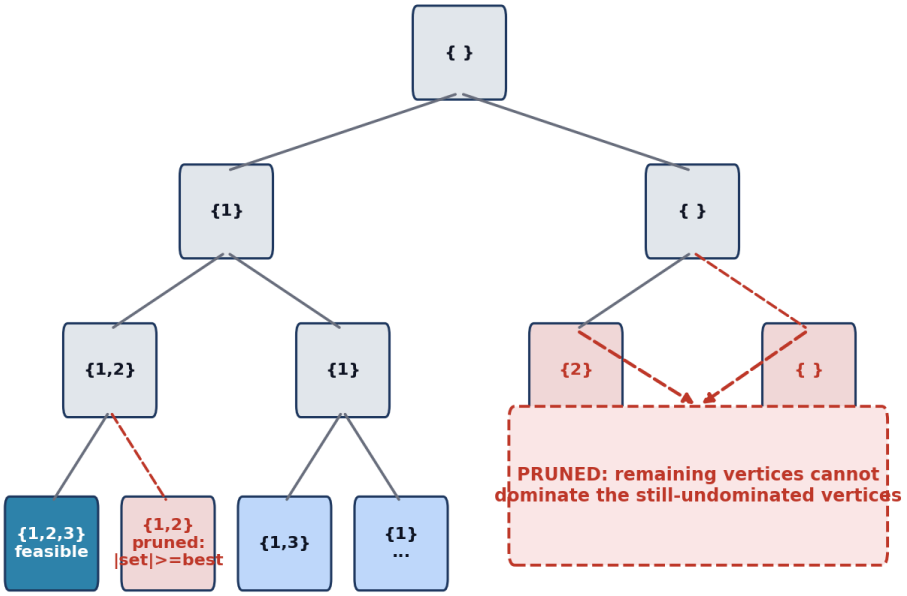


Figure 9: Pruning applied to the backtracking search tree. Dashed red edges and shaded nodes indicate branches eliminated by Rules 1 and 2.

6.5 Effect of Pruning on the Search Space

Without any pruning, the algorithm must in the worst case examine all 2^n candidate subsets. With pruning, large subtrees that cannot contain an improving solution are cut before they are ever expanded, so the number of nodes actually visited in practice is typically far smaller than 2^n. It is important to state precisely what pruning does and does not achieve: pruning improves the practical, average-case running time by eliminating unpromising branches early; it does not change the worst-case exponential nature of the problem, which remains O(2^n) in the theoretical worst case (Section 9).

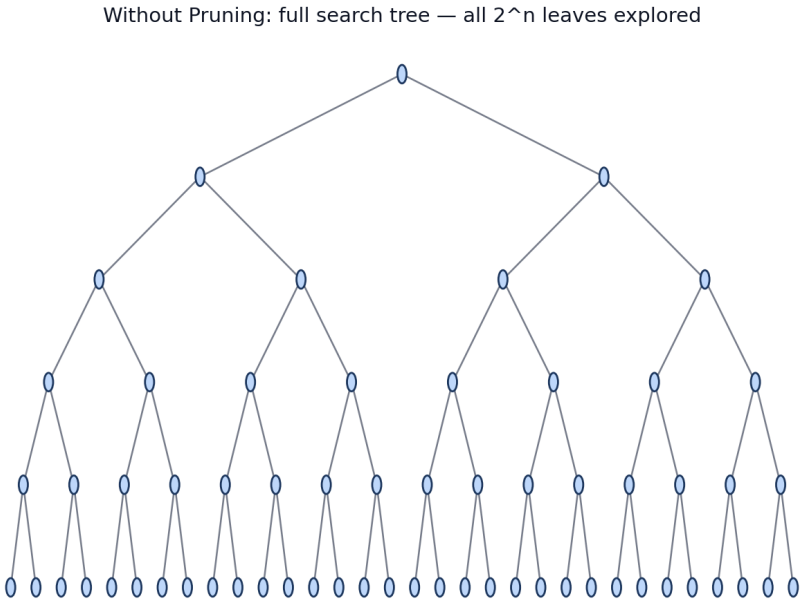


Figure 10: Search tree without pruning — every branch is explored down to a complete assignment.

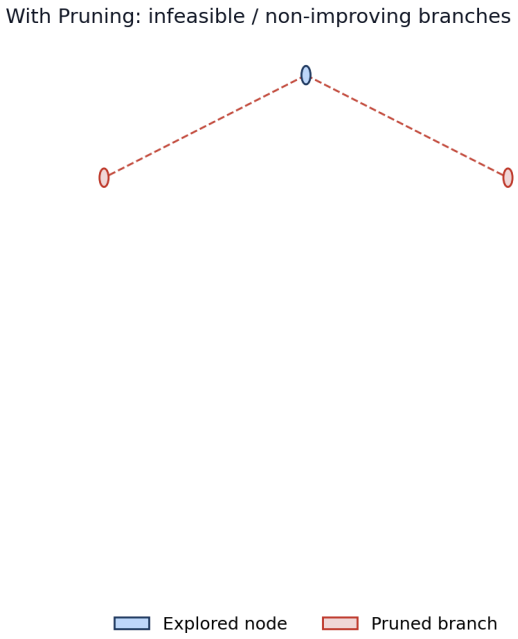


Figure 11: The same search tree with pruning applied — infeasible and non-improving branches (light shading, dashed edges) are cut early.

Rule	Condition	Purpose
Rule 1	|selectedSet| ≥ |bestSolution|	Stop branches that already match/exceed the best size
Rule 2	Remaining vertices cannot cover undominated set	Stop branches that cannot become feasible
Rule 3	All vertices already dominated	Accept early and stop over-including vertices
Rule 4	size + lower bound ≥ best	Stop branches that provably cannot beat the best

Table 5: Pruning Conditions Used in the Backtracking Search.

7. Worked Example

Consider the graph G = (V, E) with V = {1,2,3,4,5,6,7} shown in Figure 12. This example is deliberately small enough to trace by hand while still exhibiting a non-trivial minimum dominating set.

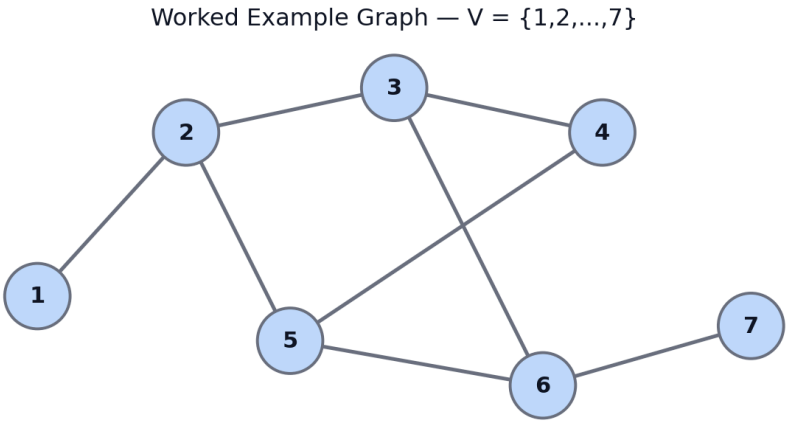


Figure 12: Worked example graph on 7 vertices.

7.1 Step-by-Step Backtracking Trace

Step 1. Start with the empty candidate set, D = ∅; all seven vertices are undominated.

Step 2. Select vertex 1 (include branch). This dominates {1, 2} via the edge (1,2).

Step 3. Vertices 3, 4, 5, 6, 7 remain undominated after this decision.

Step 4. Continue the include/exclude search over vertices 2..7. Selecting vertex 2 next would only add coverage of {1,3,5}, still leaving 4,6,7 undominated, and Pruning Rule 2 shows this path cannot reach the current best bound as efficiently as selecting vertex 3.

Step 5. Select vertex 3 (include branch). Vertex 3 is adjacent to {2,4,6}, so its closed neighbourhood dominates {2,3,4,6}. Combined with vertex 1's coverage, the dominated set becomes {1,2,3,4,6}; only vertices 5 and 7 remain undominated.

Step 6. Select vertex 6 (include branch). Vertex 6 is adjacent to {3,5,7}, so its closed neighbourhood dominates {3,5,6,7}. The accumulated dominated set is now {1,2,3,4,5,6,7} — every vertex is covered.

Step 7. Pruning Rule 3 (early acceptance) fires: the current selection already dominates all vertices, so the search accepts D = {1,3,6}, records |D| = 3 as the best solution so far, and does not explore adding any further vertex along this branch.

Step 8. The remaining branches of the search tree are explored under Rules 1 and 2; no branch is found that dominates all seven vertices with fewer than 3 selected vertices, so the search concludes that {1,3,6} is optimal.

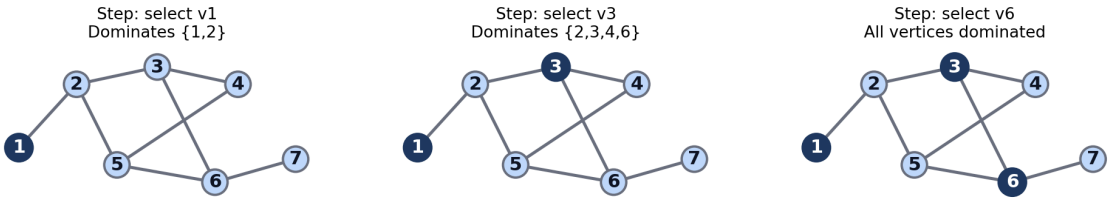


Figure 13: State progression of the worked example — the set of dominated vertices grows as vertices 1, 3, and 6 are selected in turn.

7.2 Trace Table

Step	Selected Set	Dominated Vertices	Undominated Vertices	Action
1	{ }	{ }	{1,2,3,4,5,6,7}	Initialize search
2	{1}	{1,2}	{3,4,5,6,7}	Include vertex 1
5	{1,3}	{1,2,3,4,6}	{5,7}	Include vertex 3
6	{1,3,6}	{1,2,3,4,5,6,7}	{ }	Include vertex 6 — feasible
7	{1,3,6}	{1,2,3,4,5,6,7}	{ }	Accept; update best solution
8	{1,3,6}	{1,2,3,4,5,6,7}	{ }	No smaller feasible set found — optimal

Table 6: Backtracking Search Steps for the Worked Example.

7.3 Final Answer

Minimum Dominating Set D = {1, 3, 6}

Minimum Size |D| = 3

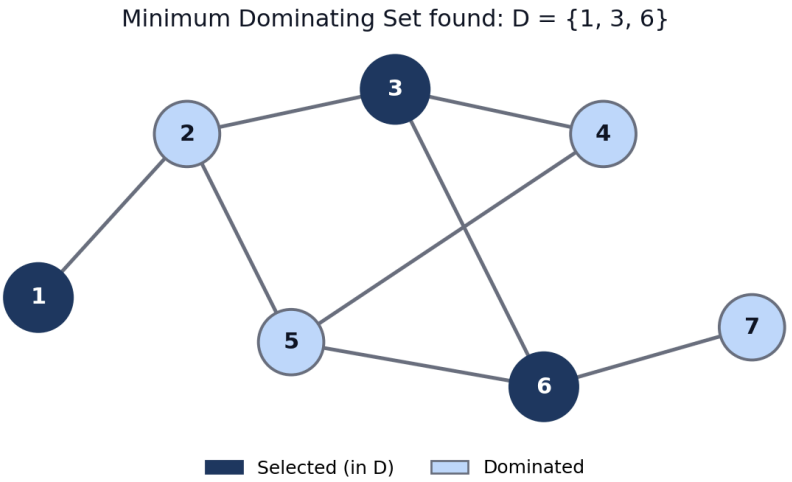


Figure 12b: The worked-example graph with the minimum dominating set D = {1, 3, 6} highlighted.

8. Correctness Proof

Theorem. The backtracking algorithm of Section 5, augmented with the pruning rules of Section 6, returns a minimum dominating set of G.

8.1 Proof

9. At every vertex, the algorithm explicitly considers both possibilities — include the vertex in D, or exclude it — via the two recursive branches of BACKTRACK.
10. Consequently, every one of the 2^n subsets of V corresponds to exactly one root-to-leaf path of the unpruned search tree; no candidate subset is skipped by construction.
11. A candidate is only accepted as a solution when isDominating(selectedSet) evaluates to true, i.e., when every vertex of V lies in the closed neighbourhood of the selected set. Therefore every candidate that reaches acceptance is, by definition, a valid dominating set.
12. The algorithm maintains a variable bestSolution that is updated only when a newly accepted feasible set is strictly smaller than the current bestSolution. By induction over the order in which feasible leaves are reached, bestSolution always holds the smallest feasible set discovered so far.
13. It remains to show that pruning never discards a branch that could contain a smaller feasible solution than the eventual bestSolution. Pruning Rule 1 discards a branch only when |selectedSet| ≥ |bestSolution| already, and since including further vertices cannot decrease |selectedSet|, no completion of that branch can beat the current best. Pruning Rule 2 discards a branch only when it has been shown that no assignment of the remaining undecided vertices can cover the currently undominated vertices — so no completion of that branch is even feasible, let alone optimal. Pruning Rule 3 does not discard information: it accepts the current feasible set immediately, which is always at least as good as continuing to add vertices. Pruning Rule 4 discards a branch only when a valid lower bound on the additional vertices required proves the branch cannot beat bestSolution.
14. Because every pruning condition is a proof that the discarded branch cannot yield an improved feasible solution, the set of leaves that are never explored is exactly the set of leaves that could not have improved on bestSolution. The search therefore still considers every leaf capable of producing a smaller feasible set.

15. Consequently, when the search terminates, bestSolution is a valid dominating set (by step 3) of the smallest size among all dominating sets of G (by steps 4–6). This establishes that bestSolution is a minimum dominating set, completing the proof.

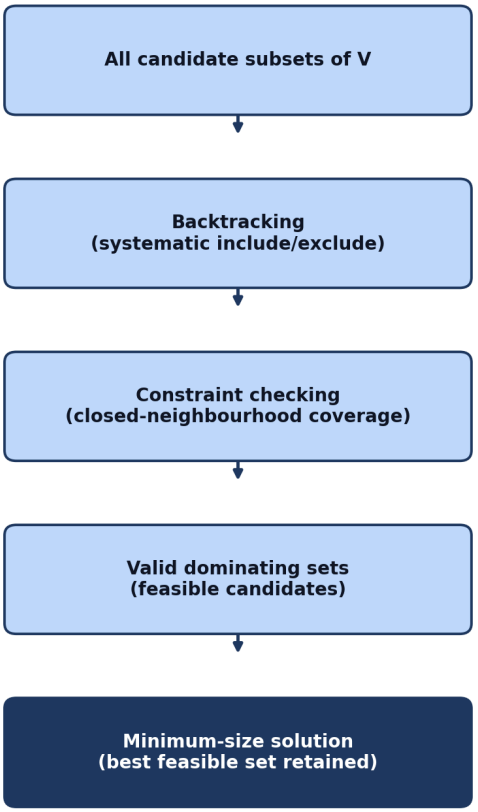


Figure 14: Correctness flow — from the full candidate space, through backtracking and constraint checking, to the minimum-size feasible solution.

8.2 Why Each Pruning Rule Preserves Correctness

Each pruning rule presented in Section 6 is a conservative, provable statement about the impossibility of improvement, never a heuristic guess. Rule 1 relies on the monotonic fact that the size of a partial selection cannot decrease as more vertices are added. Rule 2 relies on the monotonic fact that the set of vertices reachable by the remaining candidate vertices can only shrink relative to what full inclusion would achieve, so if even full inclusion cannot cover the undominated set, no sub-selection can either. Rule 3 exploits the observation that a feasible set can never be improved by adding more vertices. Rule 4 relies on a valid combinatorial lower bound on the vertices still required. Because each rule is proven rather than approximate, the algorithm remains exact: it always returns the true minimum dominating set.

9. Complexity and NP-Complete Analysis

9.1 Time Complexity

Without pruning, the backtracking algorithm explores the full binary decision tree over n vertices, which has 2^n leaves, one per candidate subset. At each leaf (and, in an unoptimized implementation, potentially at each internal node), a feasibility check costs O(n + m), where m = |E|, since it inspects each selected vertex's closed neighbourhood. The resulting worst-case time complexity is therefore approximately:

$$O(2^n \times (n + m))$$

This exponential bound is unavoidable in the worst case for any known exact algorithm for this problem, because the Dominating Set Problem is NP-Complete (Section 9.3): unless P = NP, no algorithm can solve every instance in time polynomial in n and m.

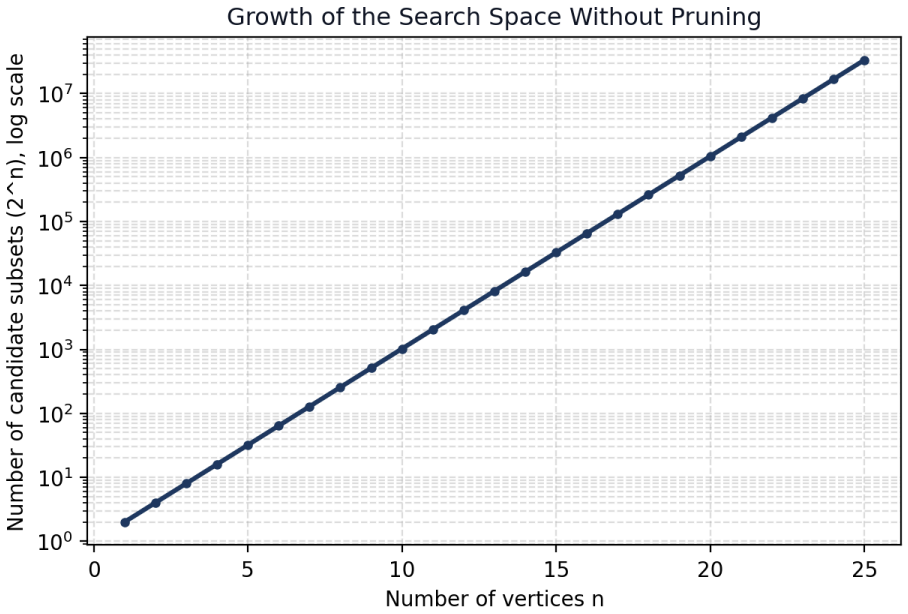


Figure 15: Growth of the number of candidate subsets (2^n) as the number of vertices n increases, shown on a logarithmic scale.

9.2 Space Complexity

The space required by the recursive backtracking procedure, excluding the storage needed for the graph itself (its adjacency structure) and implementation-specific bookkeeping, is dominated by the recursion stack and the state carried at each level. Since the recursion depth is at most n (one call per vertex decision), and each level stores a constant amount of additional state (the current selection bit-vector and the set of dominated vertices, both representable in O(n) bits), the space complexity is:

$$O(n)$$

9.3 NP, NP-Hard, and NP-Complete

These three complexity classes are frequently confused and must be distinguished precisely.

Class	Definition
NP	The class of decision problems for which a proposed solution (a certificate) can be verified in polynomial time by a deterministic algorithm.
NP-Hard	The class of problems that are at least as hard as every problem in NP, in the sense that every NP problem can be reduced to them in polynomial time. NP-Hard problems need not themselves be decision problems and need not be in NP.
NP-Complete	The intersection of NP and NP-Hard: problems that are both verifiable in polynomial time and at least as hard as every other problem in NP.

Table 8: NP versus NP-Hard versus NP-Complete.

9.4 The Dominating Set Decision Problem

To analyse the Dominating Set Problem within this framework, it is standard to consider its decision version:

Given a graph G and an integer k, does G have a dominating set of size at most k?

This decision problem belongs to NP: given a proposed set D of size at most k as a certificate, verifying that D is a valid dominating set requires only checking, for every vertex, that it lies in D or is adjacent to a member of D — a computation that takes O(n + m) time, which is polynomial in the size of the input.

The Dominating Set Decision Problem is additionally known to be NP-Complete: it is in NP (as just argued), and it has been shown, via a polynomial-time reduction from the well-known NP-Complete Vertex Cover Problem (and, in different classical treatments, from Set Cover), that every instance of an already-known NP-Complete problem can be transformed in polynomial time into an equivalent instance of the Dominating Set Decision Problem. This places it among the canonical NP-Complete problems of graph theory.

9.5 Decision versus Optimization Version

Version	Question	Complexity Class
Decision	Does a dominating set of size ≤ k exist?	NP-Complete
Optimization	Find the minimum dominating set (its exact size and members)	NP-Hard

Table 7: Complexity Analysis — Decision versus Optimization Versions, and Their Time/Space Bounds.

The optimization version is NP-Hard rather than NP-Complete because, strictly, NP-Completeness is a property of decision problems (those with a yes/no answer that can be verified in polynomial time); an optimization problem does not itself have a polynomial-time-verifiable certificate in the same sense, but it is at least as hard as its decision version, since a fast solution to the optimization version would immediately answer the decision version for any k. This is why the two versions are labelled differently in Table 7, and it is a distinction the report deliberately preserves rather than blurring.

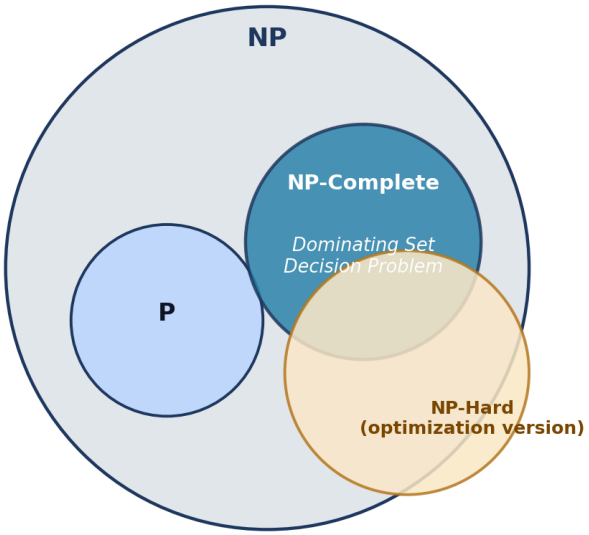


Figure 16: Relationship between P, NP, NP-Hard, and NP-Complete, with the Dominating Set Decision Problem located inside NP-Complete and the optimization version placed in NP-Hard.

10. Applications, Limitations, Comparison, and Conclusion

10.1 Real-World Applications

The Dominating Set Problem models any situation in which a small collection of locations, agents, or nodes must be chosen so that their combined coverage reaches an entire network. Selected applications include:

- Wireless sensor networks — choosing a minimal subset of sensor or relay nodes that can communicate directly with (cover) every other node in the network.
- Network monitoring — placing monitoring probes so that every link or node in a computer network is observed by at least one probe or its neighbour.
- Facility placement — siting a minimal number of facilities (e.g., fire stations, warehouses) so every location is served by, or adjacent to, a facility.
- Social network analysis — identifying a small set of highly connected individuals whose direct social ties reach the entire network, useful for information dissemination studies.
- Communication networks — selecting a minimal backbone of routing nodes that keeps every other node within one hop of the backbone.
- Surveillance systems — positioning a minimum number of cameras or sensors so every location is within observation range of at least one device.
- Infrastructure planning — minimizing the number of access points, substations, or hubs required to reach every point of a service area.
- Resource placement — general placement of scarce resources so that coverage requirements are met with the fewest resources possible.

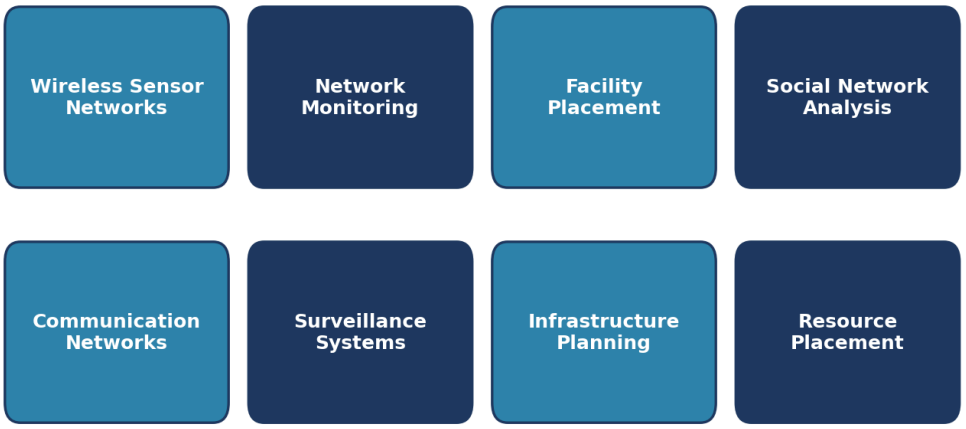


Figure 17: Representative application domains of the Dominating Set Problem.

10.2 Limitations of the Exact Approach

- The backtracking search has an exponential worst-case running time, O(2^n × (n+m)), which becomes impractical as n grows.
- Scalability is poor on large, sparse or dense graphs alike, since the theoretical worst case does not depend on graph structure.

- The size of the search space grows purely as a function of the number of vertices, regardless of how "easy" a particular instance may look.
- Exact optimization therefore becomes difficult, and in practice infeasible, once graphs reach into the hundreds or thousands of vertices.

Pruning substantially improves the practical running time on many instances by eliminating large infeasible or non-improving portions of the search tree before they are expanded. However, it is important to restate clearly: pruning does not change the fundamental worst-case exponential nature of the problem. In adversarially constructed instances, the number of nodes actually visited can still approach 2^n.

10.3 Alternatives for Large Instances

- Approximation algorithms — for example, a greedy set-cover-style approximation guarantees a solution within a logarithmic factor (O(ln n)) of optimal in polynomial time.
- Greedy heuristics — repeatedly select the vertex that dominates the largest number of currently undominated vertices, offering fast, reasonable solutions with no optimality guarantee.
- Integer programming (ILP) formulations — solved with general-purpose ILP solvers, which use their own branch-and-cut methods and can scale further than naive backtracking on structured instances.
- Branch and Bound — a more refined exact technique than plain backtracking, using tighter bounding functions to prune more aggressively while remaining exact.
- Metaheuristics — simulated annealing, genetic algorithms, and local search methods that search for good (but not certified optimal) solutions on very large graphs.

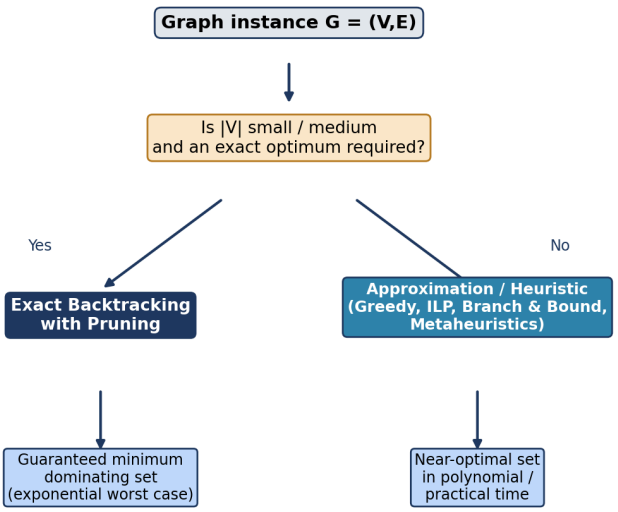


Figure 18: A practical framework for choosing between exact backtracking and approximate methods, depending on graph size and optimality requirements.

10.4 Comparison of Approaches

Approach	Exact?	Worst-Case Behaviour	Scalability	Typical Use
Brute-force backtracking	Yes	O(2^n × (n+m))	Small graphs only	Baseline / small instances
Backtracking + pruning	Yes	O(2^n × (n+m)) worst case	Small–medium graphs	Exact solutions where feasible

Approach	Exact?	Worst-Case Behaviour	Scalability	Typical Use
Branch and Bound	Yes	Exponential worst case	Medium graphs	Tighter exact search
ILP solvers	Yes	Exponential worst case (NP-Hard)	Medium–large, structure-dependent	Exact solutions via solver tooling
Greedy heuristic	No	Polynomial, O(ln n)-approx.	Large graphs	Fast, practical near-optimal sets
Metaheuristics	No	Polynomial per iteration	Very large graphs	Large-scale practical deployment

Table 9: Comparison of Exact and Approximate Approaches to the Dominating Set Problem.

10.5 Conclusion

The Dominating Set Problem is a fundamental graph-covering optimization problem: it asks for the smallest subset of vertices whose closed neighbourhoods jointly cover the entire vertex set. This report developed the problem from its formal mathematical definition through the constraints that characterize feasible solutions, an exact backtracking algorithm built on binary include/exclude decisions, four pruning rules that provably eliminate unproductive branches without sacrificing correctness, a fully worked example, a rigorous correctness proof, and a complexity analysis showing an unavoidable exponential worst case rooted in the problem's NP-Completeness.

The central insight for exact algorithms on NP-Complete problems, illustrated throughout this report, is that pruning and careful search-tree design can make exact solutions practical on real-world-sized small and medium instances, even though no pruning strategy can alter the theoretical exponential worst case. For graphs beyond this practical range, the report has outlined approximation algorithms, greedy heuristics, integer programming, branch and bound, and metaheuristic methods as the standard alternatives, each trading a guarantee of optimality for improved scalability.